Ртищева Алена Сергеевна
Группа М08-501НД
Науки о данных

Домашнее задание 4. Dockerfile по правилам

1. Создание Dockerfile для web-/ML-приложения

В качестве основы взято приложение, разработанное в ДЗ 1 на базе современного фреймворка FastAPI.

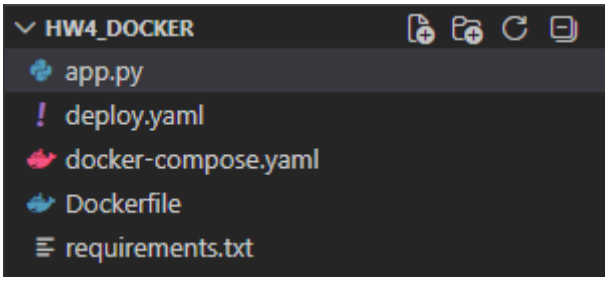


Рисунок 1 – Создание папки с файлами

```python
app.py > ...
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import Response
from pydantic import BaseModel
from typing import List, Optional
import pandas as pd
import datetime
import uvicorn
import random

# Создание приложения
app = FastAPI(title="API веб-сервера товаров (Inventory ML-App)")

# --- Блок подготовки базы данных
data = [['Антифриз EURO G11 (-45°C) зеленый, силикатный 5кг', 1025, 329, 11, 'c', 'антифриз', datetime.datetime(2026, 10, 16, 12, 36, 22)],
        ['Антифриз готовый фиолетовый Синтек MULTIFREEZE 5кг', 250, 315, 38, 'b', 'антифриз', datetime.datetime(2025, 12, 11, 8, 25, 31)],
        ['Антифриз G11 зеленый', 120, 329, 61, 'b', 'антифриз', datetime.datetime(2025, 6, 15, 15, 36, 30)],
        ['Антифриз Antifreeze OEM China OAT red -40 5кг', 390, 504, 65, 'c', 'антифриз', datetime.datetime(2025, 11, 30, 4, 12, 39)],
        ['Антифриз G11 зеленый', 135, 407, 93, 'b', 'антифриз', datetime.datetime(2026, 8, 25, 3, 24, 1)]]

df = pd.DataFrame(data)
df.columns = ['Наименование товара', 'Цена, руб.', 'cpm', 'Скидка', 'tp', 'Категория', 'dt']
df['Год'] = df['dt'].dt.year
df = df.drop(['cpm', 'tp', 'dt'], axis=1)

# Превращаем DataFrame в список словарей
items_db = []
for index, row in df.iterrows():
    items_db.append({
        "id": index + 1,
        "name": row['Наименование товара'],
        "price": row['Цена, руб.'],
        "discount": row['Скидка'],
        "category": row['Категория'],
        "year": row['Год']
    })

next_id = len(items_db) + 1
```

Рисунок 2 (начало) – Создание app.py

```python
# --- Модели данных ---
class Item(BaseModel):
    name: str
    price: int = 0
    discount: int = 0
    category: str = 'антифриз'
    year: int = 2026

class ItemUpdate(BaseModel):
    name: Optional[str] = None
    price: Optional[int] = None
    discount: Optional[int] = None
    category: Optional[str] = None
    year: Optional[int] = None

# --- Эндпоинты ---

@app.get("/")
def read_root():
    return {"status": "Active", "app": "Product API", "container": "Docker"}

@app.get("/items")
def get_all():
    return items_db

@app.get("/items/{id}")
def get_one(id: int, request: Request):
    item = next((i for i in items_db if i["id"] == id), None)
    if not item:
        raise HTTPException(404, "Товар не найден")

    # Обработка XML/JSON
    if request.headers.get('accept') == 'application/xml':
        xml = f'''<?xml version="1.0"?><item><id>{item["id"]}</id><name>{item["name"]}</name><price>{item["price"]}</price></item>'''
        return Response(content=xml, media_type="application/xml")
    return item

@app.post("/items", status_code=201)
def create(item: Item):
    global next_id
    new_item = {"id": next_id, **item.model_dump()}
    items_db.append(new_item)
    next_id += 1
    return new_item

@app.delete("/items/{id}", status_code=204)
def delete(id: int):
    global items_db
    items_db = [i for i in items_db if i["id"] != id]
    return
```

Рисунок 2 (продолжение) – Создание app.py

requirements.txt

```
fastapi
uvicorn
pandas
```

Рисунок 3 – Создание requirements.txt

Разработан Dockerfile, архитектура которого основана на принципе многостадийной сборки (Multi-stage build). Процесс разделен на два этапа:

1. Стадия подготовки (Builder).

На базе образа python:3.9-slim создается временная среда. На этом этапе копируется файл зависимостей requirements.txt и выполняется установка библиотек FastAPI, Uvicorn и Pandas. Особое внимание уделено кешированию pip-зависимостей: установка библиотек выделена в отдельный слой, который не пересобирается при изменении основного кода приложения. Использование ключа --user позволяет локализовать все установленные пакеты в директории /root/.local.

2. Стадия запуска (Runtime).

Для финального образа также используется облегченная версия python:3.9-slim. Из предыдущей стадии копируются только готовые библиотеки (COPY --from=builder), а для корректной работы сервиса прописывается переменная окружения PATH. Это исключает попадание инструментов сборки и кэшей в рабочую среду, что значительно минимизирует размер итогового образа.

Сетевое взаимодействие реализовано следующим образом. Для интеграции контейнера с внешней средой использована инструкция EXPOSE 8000. Она информирует Docker, что сервис работает на порту 8000, обеспечивая доступ к API через браузер или внешние инструменты.

Для хранения данных реализована поддержка томов с помощью инструкции VOLUME /app/data. Это создает точку монтирования, позволяющую хранить важную информацию (например, обработанные через Pandas датасеты или логи) на хост-системе, защищая ее от потери при удалении контейнера.

```
Dockerfile > ...
# Стадия 1: Builder (Сборка зависимостей)
FROM python:3.9-slim AS builder
WORKDIR /app

# Копируем файл с зависимостями приложения
COPY requirements.txt .

# Установка библиотек с использованием монтируемого кеша (требование задания)
# Флаг --user позволяет изолировать пакеты в папке пользователя /root/.local
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --user -r requirements.txt

# Стадия 2: Runtime (Финальный образ)
FROM python:3.9-slim
WORKDIR /app

# Информируем Docker о порте приложения и создаем точку монтирования для данных
EXPOSE 8000
VOLUME /app/data

# Переносим только установленные библиотеки из builder-стадии
COPY --from=builder /root/.local /root/.local

# Копируем только исполняемый код приложения
COPY app.py .

# Добавляем путь к установленным библиотекам в переменную окружения PATH
ENV PATH=/root/.local/bin:$PATH

# Запуск веб-сервера uvicorn с доступом по внешнему интерфейсу
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Рисунок 4 – Создание Dockerfile

Технически в стандартном Dockerfile невозможно жестко закрепить лимиты памяти и CPU, так как он описывает архитектуру образа, а не параметры среды его исполнения. Ограничение ресурсов настраивается на этапе запуска или оркестрации. В данной работе это реализовано декларативно в файле docker-compose.yaml (секция deploy.resources.limits), что позволяет задать границы потребления ресурсов (256MB RAM / 0.5 CPU) непосредственно при развертывании сервиса.

```yaml
docker-compose.yaml
version: '3.8'   # Версия формата Docker Compose
name: hw4-docker # Имя проекта

Run All Services
services:
  Run Service
  items_api:
    build:
      context: .                # Текущая директория как контекст сборки
      dockerfile: Dockerfile    # Файл с инструкциями по сборке образа
    container_name: hw4-docker  # Имя запущенного контейнера
    restart: always       # Перезапуск контейнера в случае сбоя
    ports:
      - "8000:8000"       # Маппинг портов: внешний 8000 на внутренний 8000
    volumes:
      - ./data:/app/data  # Связь локальной папки data с папкой в контейнере

    # Мониторинг работоспособности приложения
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/items"]
      interval: 30s       # Как часто проверять приложение
      timeout: 5s         # Время ожидания отклика
      retries: 3          # Попытки, после которых статус изменится на unhealthy

    # Управление ресурсами (требование задания)
    deploy:
      resources:
        limits:           # Верхний порог потребления (limits)
          cpus: '0.5'     # Ограничение до 0.5 CPU
          memory: 256M    # Ограничение до 256 МБ RAM
        reservations:     # Гарантированный минимум
          cpus: '0.2'
          memory: 128M
```

Рисунок 5 – Создание docker-compose.yaml

2. Сборка образа и запуск контейнера

```
PS D:\deploying ML models\hw4_docker> docker-compose up --build -d
time="2026-04-19T17:00:20+03:00" level=warning msg="D:\\deploying ML models\\hw4_docker\\docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 0.6s (13/13) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 542B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 890B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [builder 1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [builder 2/4] WORKDIR /app
 => CACHED [builder 3/4] COPY requirements.txt .
 => CACHED [builder 4/4] RUN --mount=type=cache,target=/root/.cache/pip    pip install --user --no-cache-dir -r requirements.txt
 => CACHED [stage-1 3/4] COPY --from=builder /root/.local /root/.local
 => CACHED [stage-1 4/4] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:64c2d609ae0b78fbd195f7bcf980233ea054032c21d1cfeeff11fdf8f057fcd9
 => => exporting config sha256:d0386146a84754df509af68e08e682a531113e7ae57681b5256bfab011b66d4e
 => => exporting attestation manifest sha256:9e303718e1f254c0a0d377b491eb6b8ac3459d56fba266c4c5294d8ac6e57187
 => => exporting manifest list sha256:b8cd0ab73538fedc62aba67f040daae46470f0057e0437be23d589d423a9cd38
 => => naming to docker.io/library/hw4-docker-items_api:latest
 => => unpacking to docker.io/library/hw4-docker-items_api:latest
 => resolving provenance for metadata file
[+] Running 3/3
 ✓ hw4-docker-items_api        Built
 ✓ Network hw4-docker_default  Created
 ✓ Container hw4-docker        Started
PS D:\deploying ML models\hw4_docker>
```

Рисунок 6 – Сборка образа и запуск контейнера

Поскольку я эту процедуру повторила (удалила контейнер и снова создала), Docker увидел, что файлы не менялись, и мгновенно собрал образ из кеша (CACHED), что заняло всего 0,6 секунд. Это подтверждает, что многостадийная сборка настроена эффективно.

3. Работа в docker desktop

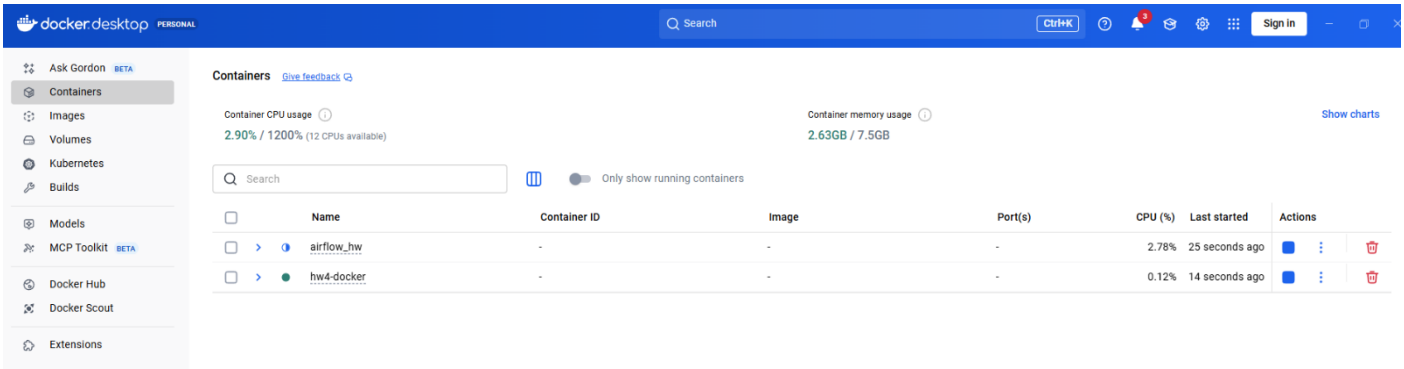


Рисунок 7 – Список контейнеров в docker desktop

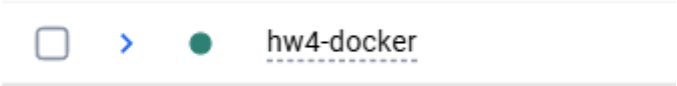


Рисунок 8 – hw4_docker

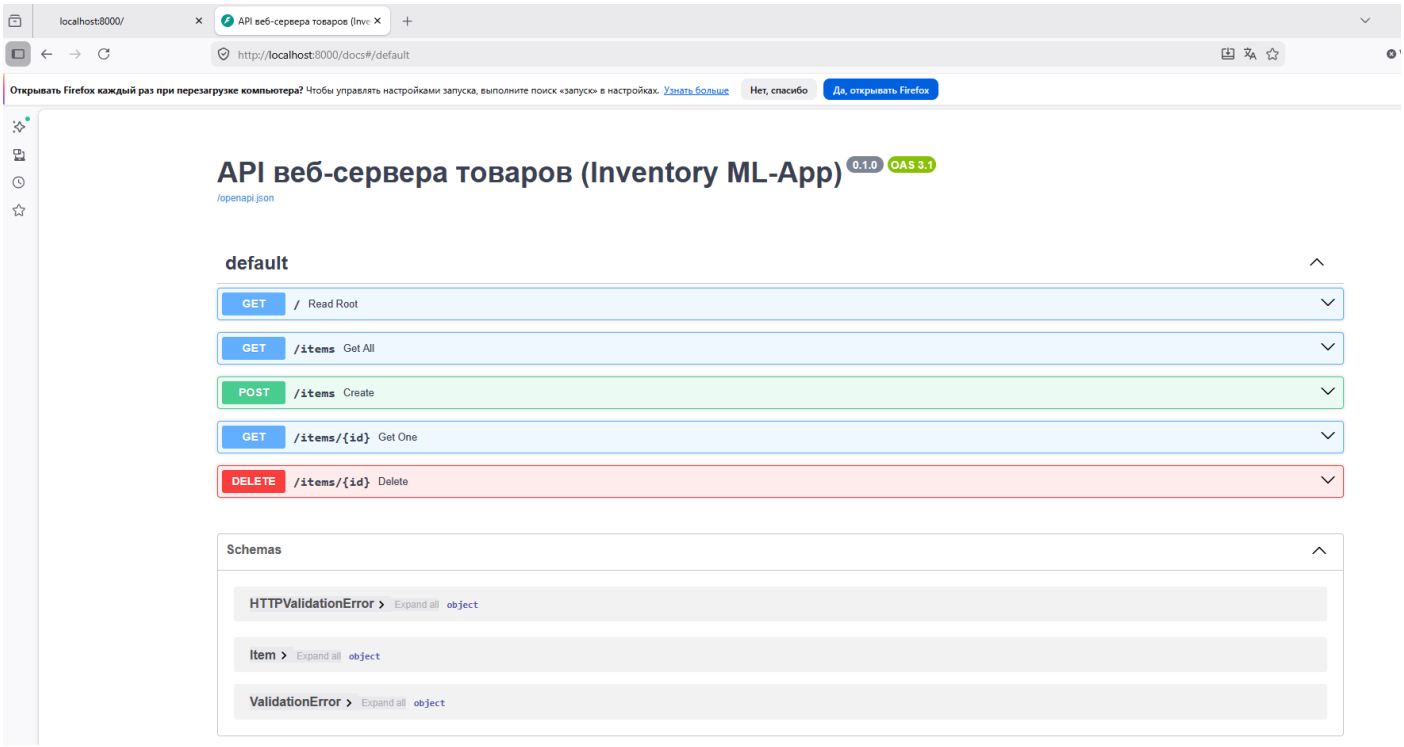


Рисунок 9 – Проверка документации (http://localhost:8000/docs#/default)

Рисунок 10 – Проверка основного списка (http://localhost:8000/items)

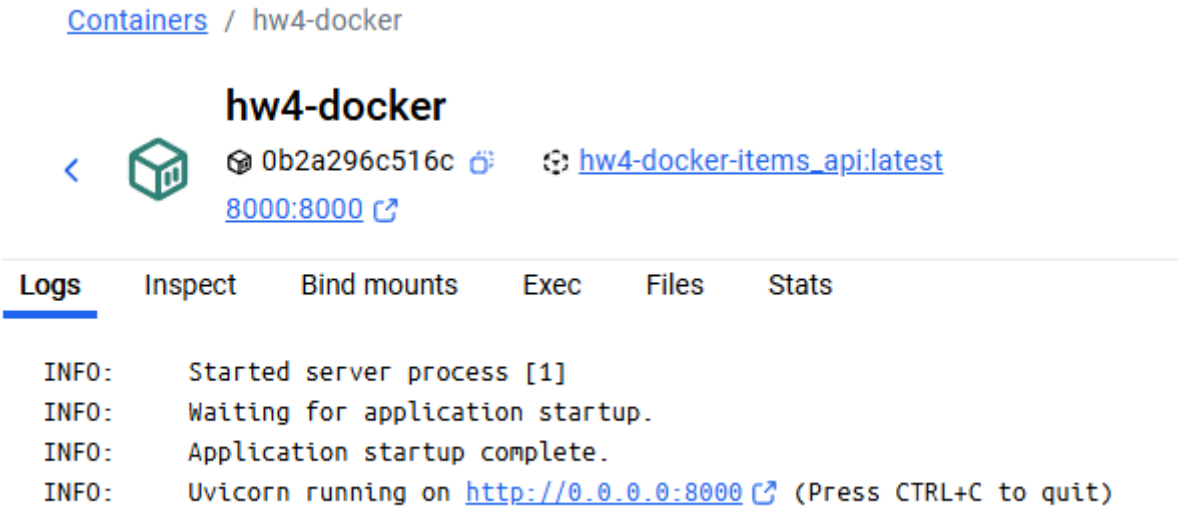


Рисунок 11 – Логи контейнера подтверждают успешный запуск сервера Uvicorn и обработку входящих HTTP-запросов к API товаров

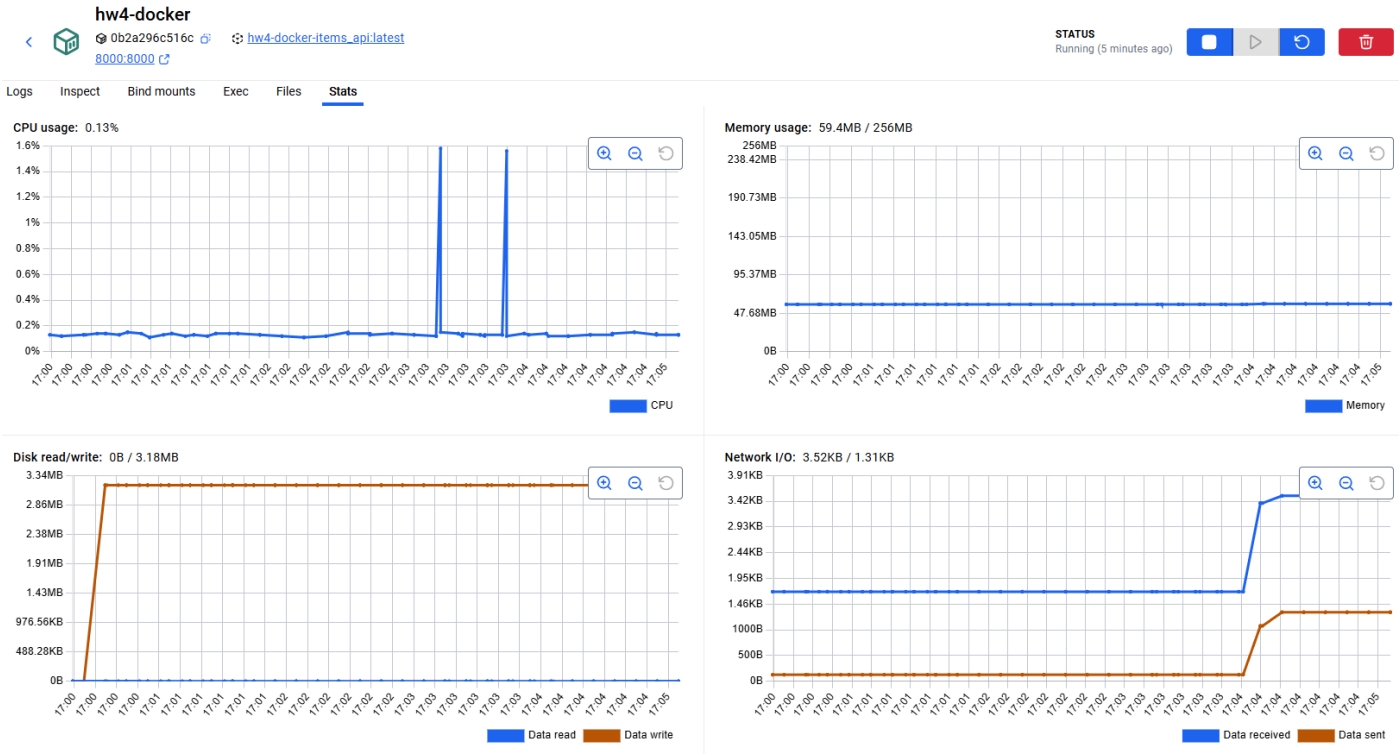
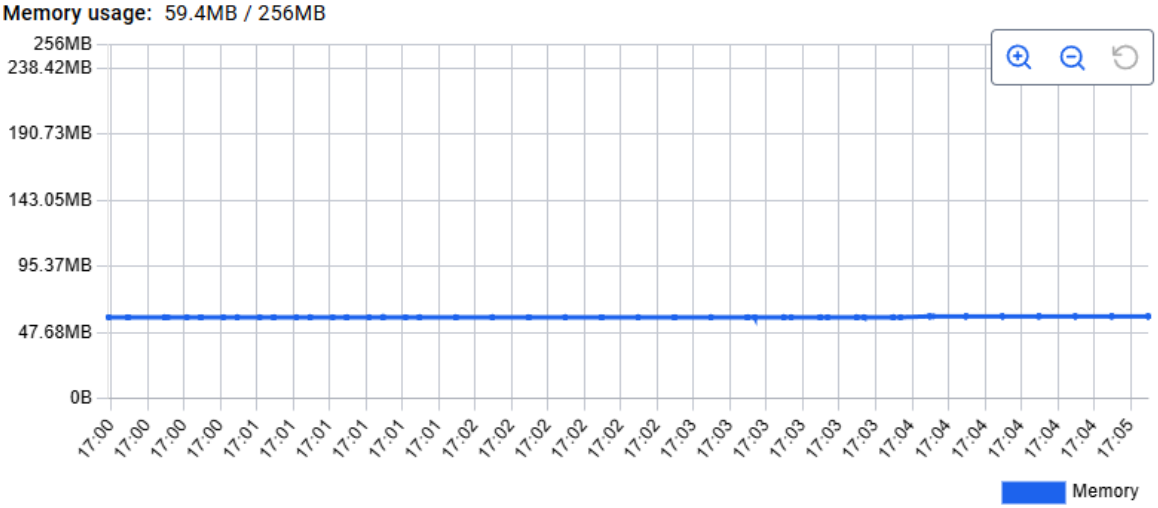
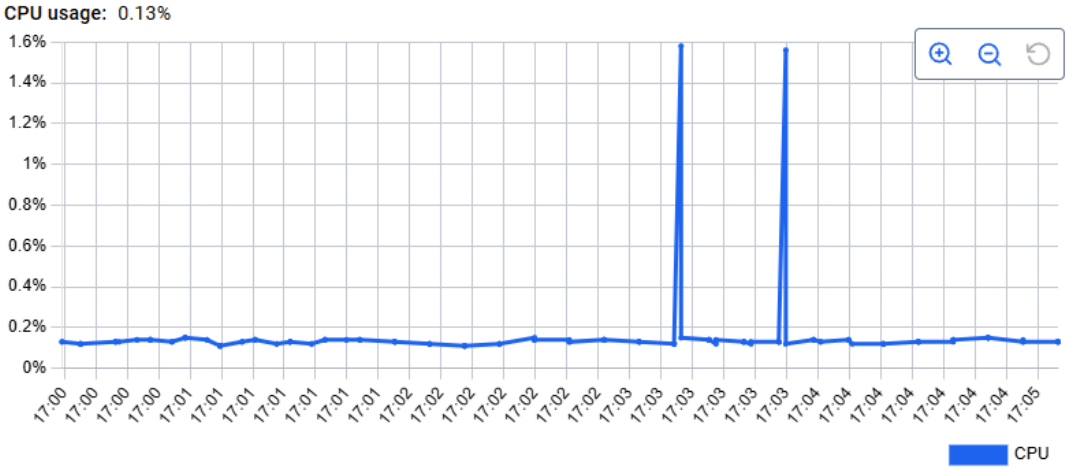


Рисунок 12 – Вкладка Stats



а)



б)

Рисунок 13 – Подтверждение лимитов памяти и CPU

4. Деплой в Kubernetes

```yaml
! k8s-deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hw4-docker-deployment   # Уникальное имя деплоймента в кластере
spec:
  replicas: 1                   # Количество запускаемых копий
  selector:
    matchLabels:
      app: hw4-docker           # Связка деплоймента с подами
  template:
    metadata:
      labels:
        app: hw4-docker         # Метка, которую получит контейнер при запуске
    spec:
      containers:
      - name: hw4-docker        # Имя контейнера внутри пода
        image: items-api:latest  # Ссылка на собранный Docker-образ
        imagePullPolicy: IfNotPresent # Использовать локальный образ, если он есть
        ports:
        - containerPort: 8000   # Порт, на котором работает FastAPI внутри контейнера

        # Выполнение обязательного условия по ограничению ресурсов
        resources:
          limits:
            memory: "256Mi"     # Максимально допустимый объем памяти
            cpu: "500m"         # Максимально допустимая нагрузка на CPU (0.5 ядра)
          requests:
            memory: "128Mi"     # Гарантированный резерв памяти
            cpu: "200m"         # Гарантированный резерв CPU

---
# Описание сетевого сервиса для доступа к приложению
apiVersion: v1
kind: Service
metadata:
  name: hw4-docker-service
spec:
  selector:
    app: hw4-docker             # Перенаправляет трафик на поды с этой меткой
  ports:
    - protocol: TCP
      port: 80                  # Внешний порт, который откроет облако
      targetPort: 8000          # Порт внутри контейнера, куда пойдет трафик
  type: LoadBalancer            # Автоматическое создание внешнего IP в облаке
```

Рисунок 14 – Создание k8s-deploy.yaml

Для перевода приложения из режима локальной разработки в режим промышленной эксплуатации был подготовлен файл-манифест k8s-deploy.yaml, который является декларативной инструкцией для системы управления контейнерами (Kubernetes). В нем четко определено, как именно оркестратор должен развертывать сервис, какие лимиты ресурсов ему необходимо выделить

для стабильной работы и как обеспечить безопасный доступ пользователей через интернет с помощью балансировщика нагрузки (LoadBalancer).

Выводы:

Docker позволяет один раз описать все настройки окружения в Dockerfile, а затем развертывать этот образ на любом сервере без ручной установки зависимостей. Многостадийная сборка оказалась эффективным инструментом для оптимизации: на первой стадии подготавливается среда сборки, а на вторую копируются только необходимые для работы файлы. Кеширование слоёв заметно ускоряет процесс – при неизменных зависимостях Docker не тратит время на их повторную установку.

Docker Compose значительно упрощает локальную разработку, позволяя в одном файле задать сети, тома, лимиты ресурсов и проверки работоспособности (healthcheck). В свою очередь, Kubernetes предоставляет мощные механизмы для промышленной эксплуатации.

Что касается автоматизации с помощью Terraform или OpenTofu, для небольших учебных проектов такой подход может казаться избыточным, так как ручное управление на начальных этапах прозрачнее. Однако при масштабировании инфраструктуры и командной разработке автоматизация становится критически важной. В практической деятельности целесообразно использовать гибридный подход: Docker Compose для разработки и тестирования, Kubernetes для стабильного продакшена, а внедрение инструментов IaC (Infrastructure as Code) рассматривать по мере роста сложности проектов.